Matthew Kolonich

1537

```sql
SELECT SUM(yearint * monthint * ordertotal)%2341 AS checksum FROM stagingTable;
```

checksum
1537

I wanted to take the 3 sales tables from 2022, 2023, and 2024 to create one table called stagingTable. To do this we first we need to upload all the data. I went to the SQL import and imported the data. I changed the files by year to pd2022, pd2023, and pd2024. PD2022 I also changed to Next I need to create the staging table. In SQL we now create a table with the following:

DROP TABLE IF EXISTS stagingTable;

CREATE TABLE stagingTable (

yearInt INT(4), monthInt INT(2), state CHAR(2), country CHAR(3),

region VARCHAR(25), customer_id INT(5), Product_Name VARCHAR(25), unitPrice INT(5),

quantityDiscount INT(5), quantity INT(6), orderTotal INT(6));

This created a table along with the 11 columns or fields that we wish to have data for.

We next need to import the data from the read file, pd2022. This data is pretty straight forward. There are 8 columns in the pd2022. The columns in pd2022 have different names, so we need pair the columns to the columns in stagingTable that they reflect. The code should look like this:

INSERT INTO stagingTable ("monthInt", "state", "country", "region", "Product_Name", "unitPrice", "quantity", "orderTotal")

SELECT "Month", "State", "Country", "Region", "Product", "Per-Unit_price", "Quantity", "Order_Total" FROM pd2022;

UPDATE stagingTable SET yearint=2022;

This added the pd2022 data to stagingTable. The columns that were not included will be marked as null. The names of the pd2022 columns are aligned with the names of the stagingTable columns to allow data to be loaded.

I next want to upload the pd2023 data. First, while loading this data, the columns do not have names. I need to change all of the column names to what they actually are. An example in changing C2 to Month. This needs to be done so the correct columns are loaded and read. We also have two quantity's sold on pd2023 because of shipping issues. We need to add both of these quantities together to get the total quantity. We also need to fill the OrderTotal line in the stagingTable. We do this by adding quantity 1 and quantity 2 and multiplying it by the Per-Unit_Price to get the total for the entire order. The code for that is:

INSERT INTO stagingTable (monthInt, region, customer_id, Product_Name, unitPrice, quantity, orderTotal)

SELECT Month, Region, Customer_ID, Product, "Per-Unit_Price", Quantity_1 + Quantity_2,

("Per-Unit_Price" * (Quantity_1 + Quantity_2))

FROM pd2023;

UPDATE stagingTable SET yearInt = 2023 WHERE yearInt IS NULL;

I next want to add the data from pd2024 to stagingTable. I do the same as pd2023 and change the name of the columns from CV to the column names used in the pd2024 table. This would be changing CV2 to Month. We then need to pair the names of the columns in pd2024 with the columns they equate to in stagingTable. There are 8 columns we need to insert. They are month, country, region, state, product, Per-Unit_Price, Quantity_Discount, Quantity, and a last column that is the order subtotal minus the quantity discount. This last column will go in the orderTotal column in the stagingTable. We also set the year to 2024 for these rows. The code is as follows:

INSERT INTO stagingTable (monthInt, state, country, region, Product_Name, unitPrice, quantityDiscount, quantity, orderTotal)

SELECT Month, State, Country, Region, Product, “Per-Unit_Price”, Quantity_Discount,

Quantity, Order_Subtotal – Quantity_Discount

FROM pd2024;

UPDATE stagingTable SET yearInt=2024

WHERE yearInt ISNULL;

We now have the table, stagingTable, that includes all the data from 2022, 2023, and 2024. We have taken the data from those tables and combined them into one that shows everything. There are only 8 columns in stagingTable and we have used functions and inserts to combine all the data into one table. The stagingTable can now be used to research and analyze all data needed for 2022, 2023, and 2024.